An AI-Enhanced Collaborative Platform for Student Mentorship and Project Management

A MINI PROJECT REPORT

Submitted by

KAVINKUMAR P	**(922523205071)**
MADHAN K	**(922523205081)**
MADHAN KUMAR R	**(922523205082)**
PERIYASAMY N	**(922523205120)**

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

V.S.B. ENGINEERING COLLEGE, KARUR-639111
(AN AUTONOMOUS INSTITUTION)

ANNA UNIVERSITY :: CHENNAI 600 025

DECEMBER 2025

ANNA UNIVERSITY :: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report titled **"An AI-Enhanced Collaborative Platform forStudent Mentorship and Project Management"** is the bonafide work of

"KAVINKUMAR P (922521205071), MADHAN K (922521205081)

MADHAN KUMAR R (922521205082) and PERIYASAMY N

(922521205120)" who carried out under my supervision.

SIGNATURE

Mr. K. MANIVANNAN, M.Tech., (Ph.D.)

HEAD OF THE DEPARTMENT

Department of Information Technology
V.S.B. Engineering College
Karur-639111.

SIGNATURE

Mr. V. S. NEGALURMATH
M.Tech.,

SUPERVISOR

Assistant Professor
Department of Information Technology
V.S.B. Engineering College
Karur-639111.

Submitted for Anna University Project Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we express our thanks to our parents for providing us with a very nice environment for doing this project. We wish to express our sincere thanks to our founder and Chairman **Shri. V. S. BALSAMY B.Sc., L.L.B.,** for his endeavour in educating us in this premier institution.

We extend our gratitude to **Dr. C. VENNILA, M.E., Ph.D.,** Principal and D**r. T. S. KIRUBHASHANKAR M.E., Ph.D.,** Vice Principal, **V.S.B. ENGINEERING COLLEGE, KARUR** for their high degree of encouragement and moral support during this project.

We express our indebtedness to **Mr. K. MANIVANNAN, M.Tech., (Ph.D.),** Head of the Department, Department of Information Technology for his guidance throughout the course of our project.

We are grateful to our Project Supervisor **Mr. V. S. NEGALURMATH M.Tech.,**

Assistant Professor, Department of Information Technology for his valuable support.

Our sincere thanks to all the faculty members of V.S.B. Engineering College and our friends for their help in the successful completion of this project work.

Finally, we bow before God, the almighty who always had a better plan for us. We give our praise and glory to almighty god for successful completion of our project

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Institution:

We endeavour to impart futuristic technical education of the highest quality to the student community and to inculcate discipline in them to face the world with self-confidence and thus we prepare them for life as responsible citizens to uphold human values and to be of service at large. We strive to bring of the Institution as an Institution of academic excellence of international standard.

Mission of the Institution:

We transform persons into personalities by the state-of the art infrastructure, time consciousness, quick response and the best academic practices through assessment and advice.

Vision of the Department:

To provide professional computing training to make competent IT engineers and thus prepare the students to work in the emerging trends of information technology field.

Mission of the Department:

1. Producing quality IT engineers with good technical knowledge by effective teaching.
2. Making competent engineers to adapt to the dynamic needs of industries.
3. Developing extensive learning skills in studies.
4. Inculcating strong ethical values and professionalism to serve society with responsibility.

Program Educational Objectives (PEOs)

PEO 1: To make graduates to be proficient in utilizing the fundamental knowledge of various streams in engineering and technology.

PEO 2: To enrich graduates with the core competencies necessary for applying knowledge of computers and technologies in the context of business enterprise.

PEO 3: To enable graduates think logically and to pursue lifelong learning to understand technical issues related to computing systems and to provide optimal solutions.

PEO 4: To enable graduates, develop hardware and software system by understanding the importance of social, business and environmental needs in the social context.

Program Outcomes (POs)

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems & design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcome (PSOs)

PSO1: Apply the mathematical and the computing knowledge to identify and provide solutions for computing problems.

PSO2: Design and develop computer programs in the areas related to algorithms, networking, web design and data analytics of varying complexity.

PSO3: Apply standard Engineering practices and strategies in software project using open- source environment to deliver a quality product for business success.

ABSTRACT

Education today requires more than just classroom teaching; it needs proper mentorship and teamwork to help students succeed. However, students often struggle to find the right mentors, come up with good project ideas, form balanced teams, and keep track of their project progress. Most existing platforms are either too general or designed for industry use and do not meet the specific needs of students. This paper introduces an AI-powered platform with two main parts: Mentor Hub and Project Hub. The Mentor Hub helps students find suitable mentors using AI recommendations, supports both one on-one and group mentoring sessions, and keeps mentor profiles updated with ratings and reviews. The Project Hub suggests project ideas based on current academic topics, forms teams based on students' skills, tracks project progress with easy-to use dashboards, and allows sharing of resources in one place. By combining mentorship and project management in one platform, this system helps students work better together, get high-quality guidance, and encourages new ideas. Using AI, it provides personalized support and smooth project handling, bridging the gap between mentorship and project work in education. After completion of this project PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PO12 and PSO1 ,PSO2 ,PSO3

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**vii**
	LIST OF FIGURES	**xii**
	LIST OF ABBREVIATIONS	**xiii**
1.	**INTRODUCTION**	**1**
	1.1 Project Overview	2
	1.2 Problem Statement	3
	1.3 Objectives	4
	1.4 Scope of the System	5
2.	**RELATED WORK / LITERATURE SURVEY**	**7**
3.	**PROPOSED SYSTEM**	**8**
	3.1 System Architecture	8
	3.2 Module Description	9
	3.3 Workflow of the System	11
	3.4 Advantages of Proposed System	12
4.	**SYSTEM SPECIFICATIONS**	**14**
	4.1 Hardware Requirements	**14**
	4.1.1 Server Requirements	14
	4.1.2 Client Devices	15
	4.2 Software Requirements	**16**
	4.2.1 Operating System	16
	4.2.2 Java Servlets & Tomcat	17
	4.2.3 MySQL Database	17
	4.2.4 Front-End Frameworks	18
	4.3 Network Requirements	**18**
	4.3.1 Server–Client Communication	19

4.3.2 Security Configuration 19
4.3.3 Data Synchronization 19
5. IMPLEMENTATION 21
5.1 Modules Used 21
5.2 Module Description 22
5.2.1 User Authentication Module 22
5.2.2 Booking / Request Management 22
5.2.3 Payment / Transaction Module 23
5.2.4 Admin Panel 23
5.2.5 Notification Module 24
5.2.6 Data Analytics Module 25
5.2.7 Performance Analysis 25
6. SYSTEM DESIGN 27
6.1 System Architecture Diagram 27
6.2 UML Diagram 28
6.3 Use Case Diagram 30
6.4 Collaboration Diagram 31
6.5 Sequence Diagram 33
6.6 Deployment Diagram 34
6.7 Data Flow Diagram 35
7. SYSTEM TESTING 38
7.1 Testing Overview 38
7.2 Types of Testing 39
7.2.1 Integration Testing 39
7.2.2 Functional Testing 40
7.2.3 System Testing 40
7.2.4 White Box Testing 41
7.2.5 Black Box Testing 41

7.2.6 Unit Testing 42
7.2.7 Acceptance Testing 42
8. FEASIBILITY STUDY 44
8.1 Feasibility Types 44
8.1.1 Technical Feasibility 44
8.1.2 Economic Feasibility 45
8.1.3 Operational Feasibility 46
8.1.4 Legal & Regulatory Feasibility 46
8.1.5 Social Feasibility 47
8.2 Methodology 48
9. CONCLUSION AND FUTURE ENHANCEMENT 49
9.1 Conclusion 49
9.2 Future Enhancement 50
APPENDIX 52
SOURCE CODE 52
SCREENSHOTS 58
REFERENCES 62
BIBLIOGRAPHY 63

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	System Architecture Diagram	8
Figure 3.2	Workflow of the System	9
Figure 3.3	Module Description Diagram	11
Figure 6.1	UML Class Diagram	27
Figure 6.2	Use Case Diagram	28
Figure 6.3	Sequence Diagram	30
Figure 6.4	Collaboration Diagram	31
Figure 6.5	Deployment Diagram	33
Figure 6.7	Data Flow Diagram	37

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	ArtificialIntelligence
DBMS	Database Management System
UML	Unified Modeling Language
DFD	Data Flow Diagram
API	Application Programming Interface
SQL	Structured Query Language

INTRODUCTION

CHAPTER 1
INTRODUCTION

Education in the modern era is undergoing a rapid digital transformation. With the increasing complexity of academic workloads, project-based learning, and the need for domain-specific guidance, students today require more than traditional classroom instruction. They seek proper mentorship, structured project environments, timely feedback, and collaborative learning spaces that help them develop skills relevant to both academic and real-world challenges. However, most institutions still depend on conventional systems for mentorship and project guidance, which are often unorganized, inefficient, and difficult to scale.

Students frequently face several obstacles throughout their academic journey. Finding the right mentor who aligns with their interests, skills, and career goals is often difficult. Even after connecting with a mentor, maintaining consistent communication, scheduling sessions, and receiving structured feedback becomes challenging without a dedicated digital platform. Similarly, project development poses its own set of difficulties. Many students struggle to select meaningful project ideas, form the right team, allocate tasks, monitor deadlines, track progress, or maintain documentation in a streamlined manner. These issues often lead to misunderstandings, delays, and reduced project efficiency.

To address these challenges, our project proposes an AI-Enhanced Mentor and Project Management Platform. This platform integrates two major components—Mentor Hub and Project Hub—supported by Artificial Intelligence to automate recommendations, enhance collaboration, and ensure seamless communication. The Mentor Hub connects students with suitable mentors and enables structured one-to-one or group mentorship. The Project Hub assists students in selecting project ideas, forming balanced teams, assigning tasks, tracking progress, and uploading deliverables. By combining these features into a single unified

system, the platform aims to revolutionize the academic mentorship and project development experience.

This introduction provides an overview of the system, the challenges addressed, the objectives of the project, and the scope of the proposed solution.

1.1 PROJECT OVERVIEW

The AI-Enhanced Mentor and Project Management Platform is designed as a comprehensive digital solution that bridges the gap between students and mentors while simplifying project development. The platform consists of three interconnected components:

Mentor Hub, Project Hub, and an AI Recommendation Engine.

The Mentor Hub acts as the central environment where mentors and students interact. Students can search for mentors based on expertise, domain, or academic needs. The system uses AI algorithms to match students with mentors who best suit their skills, interests, and learning goals. Mentors can manage mentees, schedule sessions, review requests, provide feedback, and monitor mentee progress.

The Project Hub supports every stage of project development—from idea selection to final documentation. Students can generate AI-suggested project ideas based on trending technologies, research topics, and domain-specific requirements. Teams can be created manually or through AI-based clustering to ensure balanced skill distribution.

The module includes task management, milestone tracking, progress dashboards, and a centralized resource repository. At the core is the AI Recommendation System. It enhances user experience by analyzing student profiles, mentor expertise, project requirements, past activities, and feedback. This helps the platform intelligently suggest mentors, generate project ideas, form teams, and predict potential delays or challenges in tasks. Overall, the system provides a structured, transparent, and supportive environment

that improves academic outcomes, enhances mentorship quality, and prepares students for collaborative professional environments.

1.2 PROBLEM STATEMENT

Despite the importance of mentorship and project-based learning in higher education, most institutions lack a unified platform that facilitates both. Students often struggle with various issues that hinder their academic and professional growth.

1. Difficulty in Finding Suitable Mentors Many students do not have access to the right mentors who match their academic interests or career aspirations. Institutions rarely maintain centralized systems that allow structured mentor searches or requests. Even when mentors are available, their schedules, expertise, and availability are not clearly visible to students.

2. Lack of Structured Project Development Tools Project development involves multiple stages—idea selection, team formation, planning, execution, testing, and documentation. Without digital tools, students face confusion, duplication of tasks, miscommunication, and unbalanced workload distribution. Tracking deadlines becomes difficult, often resulting in incomplete or poorly executed projects.

3. Inefficient Communication Between Students and Mentors Traditional mentor-mentee communication usually takes place via unofficial channels like messaging apps or emails. This leads to inconsistent interactions, lost messages, and no formal record of session notes or feedback.

4. Difficulty in Generating Project Ideas Students often choose repetitive, outdated, or low-impact project topics due to lack of exposure to current industry trends. Without proper guidance, ideation becomes difficult, and project quality suffers. 5. Lack of Real-Time Tracking and Assessment Both mentors and students struggle to monitor progress effectively. Without dashboards or automated tracking systems, issues such as missed deadlines, unbalanced team contributions, and poor task distribution remain unnoticed.

1.3 OBJECTIVES

The primary objective of this project is to create a unified, intelligent platform that enhances mentorship and project development in academic institutions. The specific objectives include:

1. **To provide AI-driven mentor recommendations**

The system analyzes student profiles and suggests suitable mentors based on skills, interests, and academic goals, ensuring accurate and personalized matching.

2. **To simplify project ideation using Artificial Intelligence**

The platform suggests trending, domain-specific project topics by analyzing research papers, online data, and industry trends.

3. **To enable balanced and efficient team formation**

Through AI clustering techniques, teams are formed with complementary skills to promote collaboration, reduce conflicts, and improve project outcomes.

4. **To streamline communication between students and mentors**

The platform offers structured communication channels, scheduling tools, notifications, and session tracking to enhance interaction quality.

5**. To track project progress systematically**

Dashboards and milestone tracking tools help teams and mentors monitor development and identify bottlenecks.

6. **To create a centralized repository for academic resources**

Students and mentors can upload documents, reports, reference materials, and versions of project files in an organized manner.

7. **To improve academic productivity and reduce manual workload**

Automation helps mentors manage multiple mentees efficiently and supports students with reminders, deadlines, and task updates.

These objectives aim to improve both the mentorship experience and the completeness, accuracy, and professionalism of student projects.

1.4 SCOPE OF THE SYSTEM

The scope of the proposed platform is extensive and designed to meet the academic needs of students, mentors, and institutions. It includes features and functionalities that enhance collaboration, communication, project execution, and academic decision-making.

1. Users Covered by the System

- Students: Can search mentors, receive suggestions, join sessions, form teams, generate project ideas, and track project progress.
- Mentors: Can manage mentees, schedule sessions, give feedback, review project milestones, and monitor student activity.
- Administrators: Manage platform operations, verify mentor credentials, assign roles, and oversee system health.

2. Functional Scope

- AI-driven mentor recommendation
- AI-generated project ideas
- Automated team formation
- Session scheduling and reminders
- Task creation, assignment, and tracking
- Milestone monitoring with visual dashboards
- Centralized document and resource storage
- Real-time notifications and alerts
- Feedback and rating system
- Analytics for project and mentorship performance

3. Technical Scope

The system will work as a web-based application accessible from desktops, laptops, and mobile devices. It will integrate:

- Database systems for storing profiles and project information
- Cloud-based backend for scalability
- AI models for prediction, recommendation, and clustering
- APIs for integration with tools like Google Drive, GitHub, etc.

4. Future Scope

The platform can later include:

- Virtual reality classroom sessions
- Blockchain credential verification
- Industry mentor collaboration
- Automated project evaluation systems
- Career guidance recommendations

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

The field of education technology and AI-enabled mentorship has grown rapidly, particularly in terms of mentoring or matchmaking for mentorship, project-based learning, knowledge graph generation and supporting brainstorming with AI. Even with these advancements, however, there are still very important challenges in developing an integrated project, collaborative, and student-focused mentoring platform.

Education mentoring platforms have often focused on a oneto-one mentoring relationship or just assigning a mentor without a dynamic approach. In particular, Teo et al. [1] looked at mentoring in medical education and concluded that mentoring should exist within a complex adaptive system, where goals, identity, and relationships are consistently changing. While the concept of a dynamic adaptive mentor/mentee relationship is beneficial in consideration of mentoring, it does not seem that much of the current systems include this adaptive aspect in the virtual mentoring infrastructure, as most existing models are based on static methods through a one-time assignment.

In consideration of AI-supported brainstorming, Gao et al. [2] proposed Graph of AI Ideas, which supports mentoring of research topics using knowledge graphs and large language models as supporting tools to assist students. Likewise, Aytekin and Saygın, [3] proposed ACE, an educational AI tool that can build educational knowledge graphs with prerequisite relation to guide their learning. Both concepts provide helpful applications for AI to improve collaboration and structured brainstorming during academic Inquiry, but once again, do not apply to the proposed project and simply used as appendices to the project.Recent advances in retrieval-augmented generation have also permeated the area of education. Li et al. [4] have described a retrieval-based model to enhance the quality and contextual relevance of AI-generated educational resources. Such work highlights the relevance of grounding AI-generated content in reputable sources, as this is key when producing project ideas and sharing resources in collaborative situations.

PROPOSED SYSTEM

CHAPTER 3
PROPOSED SYSTEM

The proposed system was developed to address the challenges of separate mentorship and project management platforms with one cohesive platform that provides mentor matches, project idea generation, team completion, tracking progress, and sharing resources. The system framework has been developed with multiple layers for scalability, usability, as well as to provide AI functionalities.

3.1 System Architecture

User Layer

∗ The web-based and mobile-based interface for all students and mentors.

∗ Actions that can be completed include mentor search and requests, project idea generation, team collaboration, task assignment, and task progress tracking.

∗ Reporting on project events/milestones, mentor availability, and team contributions can be updated in real-time.

Application Layer

∗ Cloud-based backend, processes requests from students and mentors, stores all profiles, projects and teams management, and handles AI algorithms.

∗ Single database handles all student/mentor profiles, projects, teams, and progress data.

∗ AI algorithms run for mentor match, project idea generation, team formation, and performance analytics.

∗ APIs to external learning resources and collaboration tools (Microsoft teams, slack, etc.)

AI and Analytics Level

∗ AI modules analyze profiles/skills and past feedback to suggest optimal mentors match for students.

∗ NLP models generate project ideas based on areas of the domain trending or gaps in the literature.

Dashboards provide real-time analytics and alerting for mentors and administrators.

1) A student logs into the platform and selects the domain of interest.
2) The system retrieves student profile, mentor availability, and project requirements.
3) Mentor recommendation algorithm suggests suitable mentors.
4) Project Hub generates AI-assisted project ideas.
5) Teams are formed using AI-based clustering for balanced skills.
6) Progress tracking dashboard monitors tasks, milestones, and contributions.
7) Notifications alert students and mentors about deadlines, pending tasks, and updates.

3.2 Module Description

The Mentor Hub is an integrative pillar of the Collaborative Platform, contributing to mentorship in a formal, intelligent, and efficient manner. It serves as a hub - allowing students to request advice, mentors to manage their mentees, and AIbased algorithms to match them intelligently.

Mentorship Requests*:* Students can submit one-to-one mentorship requests for academic, technical, or career-related guidance. Each request may include details such as the student's domain, area of interest, urgency, and preferred meeting times. Mentors receive notifications for these requests and can choose to accept, schedule, or decline them, thereby ensuring structured interaction**.**

Group Mentorship Sessions*:* In addition to individual requests, mentors can organize group mentorship sessions to engage multiple students at once. These sessions are supported by automated scheduling, notification, and tracking features. Group mentorship not only reduces the mentor's workload but also enables peer-to-peer learning and collaborative discussions.

Dynamic Mentor Profiles*:* Mentor profiles evolve dynamically over time to ensure transparency and trust. Profiles are updated based on ratings and reviews provided by students, the number and duration of mentoring sessions, and areas of expertise or specialization. This

feature ensures quality-driven mentorship and helps students make informed decisions when selecting mentors.

AI-Based Mentor Recommendation*:* The system leverages artificial intelligence to recommend suitable mentors for students. Compatibility scores are calculated by analyzing skill matches between student needs and mentor expertise, mentor availability and workload, as well as previous ratings and performance. Based on these scores, students receive a ranked list of the top N mentors most relevant to their profiles.

Session Management*:* The platform provides a dedicated session management facility that enables scheduling of both one-to-one and group mentorship sessions. Automated reminders and notifications are generated for upcoming sessions, while the system also tracks attendance and session duration. Notes and highlights from each session can be stored for future reference by both mentors and students.

Feedback and Rating System*:* After each session, students can rate their mentors and provide feedback. These ratings contribute to refining mentor profiles and improving the accuracy of the recommendation algorithm. This feedback loop maintains accountability and encourages continuous improvement in mentorship quality.

Analytics Dashboard for Mentors*:* Mentors have access to a personalized analytics dashboard that provides insights such as the number of active mentees, session success rates, and average feedback ratings. These metrics allow mentors to evaluate their own effectiveness, manage workload, and optimize mentorship strategies.

Notifications and Alerts*:* The system integrates an automated notification engine to improve communication between students and mentors. Notifications include reminders for upcoming sessions, deadlines, and pending requests

Resource Sharing*:* To support effective learning, the platform offers a centralized repository for resource sharing. Mentors can upload and share documents, presentations, reference materials, or external links. This ensures easy accessibility, collaboration, and version control, enabling students to build on shared knowledge.

3.3 WORKFLOW OF THE SYSTEM

The workflow of the proposed system explains how different users interact with the platform and how the system performs each operation in a structured manner. The process ensures smooth functioning, easy navigation, and intelligent recommendations for students and mentors.

Step 1: User Registration and Login

- Students, mentors, and admin users register with their basic details.
- The system validates user credentials and provides role-based access to dashboards.

Step 2: Profile Setup

- Students update skills, interests, and academic background.
- Mentors update areas of expertise, qualifications, project domains, and availability.
- Admin verifies mentor profiles before approval.

Step 3: Mentor Recommendation

- The AI engine analyzes student data and mentor expertise.
- Based on matching accuracy, the system suggests the most suitable mentors.
- Students send mentorship requests, and mentors accept/decline them.

Step 4: Project Idea Selection

- Students can explore AI-generated project ideas based on trends and domain requirements.
- Students may choose an idea or propose their own.
- Mentors review and approve the selected project idea.

Step 5: Team Formation

- Teams can be formed manually by inviting members.
- Or the AI system forms balanced teams based on skills, experience, and compatibility.

Step 6: Task Allocation and Planning

- The team creates project tasks and assigns responsibilities.
- Milestones and deadlines are added to the project timeline.
- The system displays progress using a visual dashboard.

Step 7: Mentorship Sessions

- Students request mentoring sessions for guidance.
- Mentors schedule sessions, give feedback, and monitor progress.
- Session history is stored for future reference.

Step 8: Project Progress Monitoring

- The dashboard displays task completion percentage, pending work, and overall status.
- Students update daily/weekly progress.
- Mentors review updates and provide corrective actions.

Step 9: Resource Sharing & Documentation

- Users upload reports, presentations, code files, and reference materials.
- The system maintains version control for each upload.

Step 10: Final Evaluation

- After project completion, mentors evaluate the project based on quality, teamwork, and documentation.
- The system stores results and marks the project cycle as completed.

This structured workflow ensures efficient collaboration, transparency, and a smooth project lifecycle.

3.4 ADVANTAGES OF THE PROPOSED SYSTEM

1. AI-Based Personalization

- The system recommends the best mentor based on student profiles.
- AI suggests relevant project ideas and helps form balanced teams.

2. Improved Mentor–Student Communication

- Dedicated communication features reduce the confusion of using multiple apps.
- Mentors can schedule sessions, send reminders, and track mentee progress.

3. Efficient Project Management

- The Project Hub supports the complete project cycle: idea → planning → execution → documentation.
- Task allocation and milestone tracking help teams stay organized.

4. Real-Time Progress Tracking

- Dashboards show task updates, pending work, deadlines, and performance graphs.
- Both mentors and students can monitor project status instantly.

5. Balanced and Productive Team Formation

- AI ensures teams have a good mix of skills.
- This increases collaboration quality and reduces conflicts.

6. Centralized Resource Storage

- All documents, reports, and reference files are stored in one secure location.
- Easy retrieval prevents loss of data.

7. Reduced Manual Work for Mentors & Admins

- Automated reminders, scheduling, matching, and progress tracking save time.
- Admins can manage users and system health with minimal effort.

8. Enhanced Academic Output

- Students receive timely guidance, leading to better project quality.
- Mentors can efficiently handle multiple teams.

9. Scalable and Secure System

- The system supports multiple users without performance issues.
- Security features like authentication and encrypted data protect user information.

SYSTEM SPECIFICATIONS

CHAPTER 4

SYSTEM SPECIFICATIONS

The development of the AI-Enhanced Mentor and Project Management Platform requires a well-defined set of system specifications to ensure smooth functioning, high performance, and scalability. These specifications act as the foundation for deployment, execution, security, and maintenance of the application.
This chapter outlines the hardware, software, and network requirements essential for running the system effectively.

4.1 Hardware Requirements

Hardware specifications define the minimum and recommended configurations necessary for both server-side and client-side operations. Since the system involves real-time data processing, AI-based recommendations, database interactions, and web-based functionalities, adequate hardware resources ensure smooth execution.

4.1.1 Server Requirements

The server acts as the backbone of the platform. It handles user authentication, AI processing, database management, API requests, and resource sharing. For medium-scale deployment (500–2000 users), the following server configuration is recommended:

Minimum Server Specifications

- Processor: Intel Core i5 / AMD Ryzen 5
- Clock Speed: 2.4 GHz or higher
- RAM: 8 GB
- Storage: 250 GB SSD
- Network Interface: 1 Gbps Ethernet
- Operating System: Windows Server / Linux (Ubuntu 20.04+)

Recommended Server Specifications

- Processor: Intel Xeon / AMD EPYC
- Clock Speed: 3.0 GHz and above
- RAM: 16 GB to 32 GB
- Storage: 500 GB – 1 TB SSD (for database + AI models)
- Backup Drive: External/Cloud backup
- Graphics Support: Optional (for AI training enhancement)
- Availability: 24×7 uptime with uninterrupted power supply (UPS)

The server hosts:

- Backend services
- AI algorithms
- Databases
- File storage
- Application logic

Efficient hardware ensures fast response times, minimal downtime, and smooth multi-user access.

4.1.2 Client Devices

Client devices refer to user-end systems used by students, mentors, and administrators to access the platform. Because the system is web-based, lightweight hardware is sufficient.

Minimum Client Specifications

- Device: Desktop / Laptop / Smart Phone
- Processor: Intel Core i3 or equivalent
- RAM: 4 GB
- Storage: 10 GB free space
- Browser: Chrome, Firefox, Edge, or Safari
- Internet Connection: Stable 2–5 Mbps

Recommended Client Specifications

- Processor: Intel Core i5 or above
- RAM: 8 GB

- Display: Full HD for better user experience
- Internet: 10–20 Mbps for video mentoring sessions

These devices allow users to:

- Access dashboards
- Generate project ideas
- Join mentorship sessions
- Track progress
- Upload/download resources

4.2 Software Requirements

Software specifications define the platforms, technologies, and frameworks used to develop, deploy, and run the system. Choosing the right software ensures scalability, security, and seamless integration with AI components.

4.2.1 Operating System

The system supports multiple operating systems depending on deployment needs.

Server-Side Operating Systems

- Linux (Ubuntu 20.04 / CentOS) – Highly recommended for stability
- Windows Server 2016/2019 – For institutions preferring Microsoft ecosystem

Client-Side Operating Systems

- Windows 10 / 11
- macOS
- Linux
- Android / iOS (for mobile devices)

A cross-platform architecture ensures compatibility across multiple devices.

4.2.2 Java Servlets & Tomcat

Java Servlets form the core backend of many academic systems due to their speed and security.

Why Java Servlets?

- High performance and platform independence
- Strong security features
- Reliable for large-scale web applications

Apache Tomcat Server

The system uses Apache Tomcat 9/10 as the application server for:

- Handling HTTP requests
- Executing Java Servlets
- Managing session states
- Hosting backend APIs

Tomcat provides a lightweight and efficient environment suitable for educational platforms.

4.2.3 MySQL Database

MySQL is used as the primary relational database due to its efficiency, reliability, and scalability.

Key Features Used

- Fast query processing
- Secure data transactions
- Relationship management
- Support for large datasets
- SQL compatibility

Data Stored Includes

- User profiles (students & mentors)
- Mentor ratings and reviews
- Project details
- Team formation data
- Task assignments

- Session logs

MySQL ensures data integrity and efficient retrieval during heavy system usage.

4.2.4 Front-End Frameworks

The front-end is designed to be interactive, responsive, and user-friendly.

Technologies Used

- HTML5 – Structure
- CSS3 / Tailwind – Styling
- JavaScript – Client-side logic
- React.js / Angular – Component-based UI
- Bootstrap – Responsive design

Front-End Features

- Clean user interface
- Real-time dashboards
- Notifications and alerts
- Interactive project and mentor lists
- Mobile-friendly layout

These frameworks ensure a smooth experience across different device screens.

4.3 Network Requirements

Since the platform is web-based and relies heavily on real-time interactions, a stable network environment is essential. Network requirements ensure that users can access the system without delays, disconnections, or performance issues.

4.3.1 Server–Client Communication

Effective communication between client devices and the server ensures seamless operation.

Communication Requirements

- HTTP/HTTPS protocols
- RESTful APIs for backend interaction

- Low-latency environment (below 150 ms)
- Bandwidth: 5–20 Mbps depending on number of users

Cloud or institutional intranet ensures smooth functioning for all users.

4.3.2 Security Configuration

Security is essential to protect user data and maintain system trust.

Security Implementations

- SSL Certificates for HTTPS
- Firewall protection
- Role-based authentication (admin/mentor/student)
- Two Factor Authentication (optional)
- Data encryption for sensitive information
- Secure API Gateways
- Regular security audits

These measures protect against unauthorized access, data breaches, and cyberattacks.

4.3.3 Data Synchronization

Data synchronization ensures that changes made in one module are updated instantly across the entire system.

Synchronization Methods

- Real-time database updates
- Auto-refreshing dashboards
- WebSocket/SignalR (optional) for push notifications
- Scheduled backups
- Cloud sync (for external storage options)

Why Synchronization is Important?

- Ensures accuracy in mentor availability
- Reflects real-time project progress
- Prevents duplication of tasks

IMPLEMENTATION

CHAPTER 5

IMPLEMENTATION

The implementation phase translates the system design into a functional and operational platform. This stage involves building each module, integrating backend and frontend components, connecting databases, and ensuring that the system works as intended. The proposed AI-Enhanced Mentor & Project Management System contains multiple modules—each responsible for a specific set of operations that together create a seamless user experience.

This chapter describes the modules used in the implementation and provides detailed explanations of each module.

5.1 MODULES USED

The system is implemented using a modular architecture, where each module performs an independent function but also interacts with other modules to maintain smooth workflow. The key modules used in the system are:

1. User Authentication Module
2. Booking / Request Management Module
3. Payment / Transaction Module
4. Admin Panel Module
5. Notification Module
6. Data Analytics Module
7. Performance Analysis Module

Each module is designed to ensure data security, efficient user flow, scalability, and easy maintenance.

5.2 MODULE DESCRIPTION

This section describes the purpose, functionality, and internal behavior of each module in detail.

5.2.1 User Authentication Module

The User Authentication Module is responsible for securing the platform and managing user access. It ensures that students, mentors, and administrators are able to log in securely and access features based on their assigned roles.

Key Functions

- User registration
- Login using username/email and password
- Role-based access control (Student, Mentor, Admin)
- Password encryption using hashing
- Forgot password / password reset
- Session management and logout functionality

Workflow

1. Users enter credentials on the login page.
2. Backend verifies credentials with the database.
3. If valid, users are redirected to their respective dashboards.
4. Unauthorized users are denied access with error messages.

This module ensures that only authenticated users can access the system.

5.2.2 Booking / Request Management

This module manages mentorship requests, project booking, session scheduling, and interaction requests between students and mentors.

Key Functions

- Sending mentorship requests
- Mentor approval or rejection
- Scheduling mentor–student sessions
- Viewing past and upcoming bookings
- Managing meeting links, time slots, and availability

Workflow

1. Student selects a mentor from the recommendation list.

2. A request is sent to the mentor.
3. Mentor accepts/rejects based on availability.
4. Once approved, a session is scheduled in the calendar.
5. Notification is sent to both parties.

This module supports smooth communication and structured session management.

5.2.3 Payment / Transaction Module

(If your project includes paid mentoring or subscription model)

This module handles secure financial transactions such as premium subscriptions, mentor fees, or booking charges.

Key Functions

- Secure payment gateway integration (Razorpay/UPI/PayPal)
- Payment verification
- Transaction logs and receipts
- Refund management (if applicable)
- Admin access to payment history

Workflow

1. User selects a plan or session requiring payment.
2. Payment gateway processes transaction.
3. System verifies payment and updates records.
4. Paid features are unlocked.

This module ensures transparency and financial security.

5.2.4 Admin Panel

The Admin Panel allows administrators to oversee all system operations, user management, and platform maintenance.

Key Functions

- Manage student and mentor accounts
- Verify and approve mentor profiles

- Monitor system usage
- View logs, analytics, and reports
- Manage project categories and resources
- Handle complaints and issue reports

Workflow

1. Admin logs in using admin credentials.
2. Admin views dashboard displaying system statistics.
3. Admin performs operations such as enabling/disabling users, generating reports, or resolving issues.

This module keeps the system secure, organized, and functional.

5.2.5 Notification Module

This module ensures that users receive real-time alerts about important activities.

Key Functions

- Email notifications
- In-app alerts
- Session reminders
- Project deadline reminders
- Mentor approval/denial alerts
- Payment confirmations

Workflow

1. A trigger event occurs (request, approval, new task, etc.).
2. The system generates a notification.
3. Notification is delivered via email or app.

The notification module helps maintain timely communication and prevents missed deadlines.

5.2.6 Data Analytics Module

This module analyzes various system parameters and delivers insights to students, mentors, and administrators.

Key Functions

- Student performance analytics
- Mentor activity dashboard
- Project progress analytics
- Graphs and charts for better understanding
- User engagement statistics

Workflow

1. System collects task and activity data in real time.
2. Data is processed using analytical algorithms.
3. Graphs, charts, and reports are generated for users.

Analytics supports decision-making and improves the learning experience.

5.2.7 Performance Analysis

This module ensures the system performs effectively even under heavy usage. It evaluates speed, scalability, and resource utilization.

Key Functions

- Server performance tracking
- Response time monitoring
- Load and stress testing
- Identifying performance bottlenecks
- Database optimization

Workflow

1. The system collects usage statistics such as number of users, queries, and API calls.
2. Performance metrics are analyzed.
3. Improvements are applied to maintain smooth functioning.

SYSTEM DESIGN

CHAPTER 6

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces, and data flow that together form the complete software system. In this chapter, the platform's structure is represented through multiple diagrams that illustrate how the system works internally.

The design ensures efficiency, reliability, scalability, and smooth user interaction across all modules.

6.1 SYSTEM ARCHITECTURE DIAGRAM

The System Architecture Diagram represents the high-level structure of the application. It shows the major components and how they interact with each other.

Description

The system follows a three-tier architecture:

1**. Presentation Layer (Front-End)**

- Built using HTML, CSS, JavaScript, React/Angular
- Displays dashboards for students, mentors, and admins
- Handles user input, navigation, and UI rendering

2. **Application Layer (Back-End)**

- Developed using Java Servlets
- Contains business logic
- Handles mentor matching, project idea generation, session scheduling, and task management
- Communicates with the database and AI engine

3. **Data Layer (Database)**

- MySQL database stores user profiles, projects, tasks, session details, notifications, and analytics

Architecture Flow

User → Browser → Application Server → Database → AI Engine → Response

Everything flows through REST APIs and follows secure HTTPS communication.

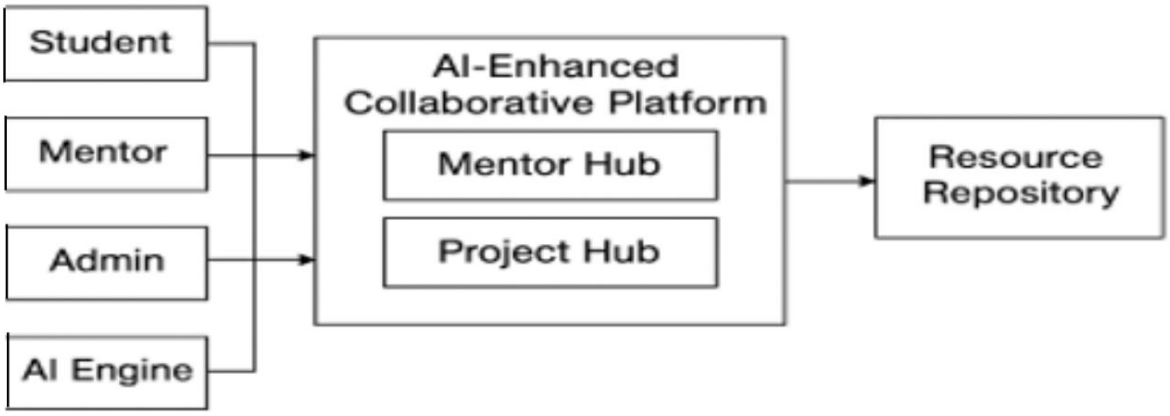


Figure 3.1: Block diagram of the proposed system

6.2 UML DIAGRAM

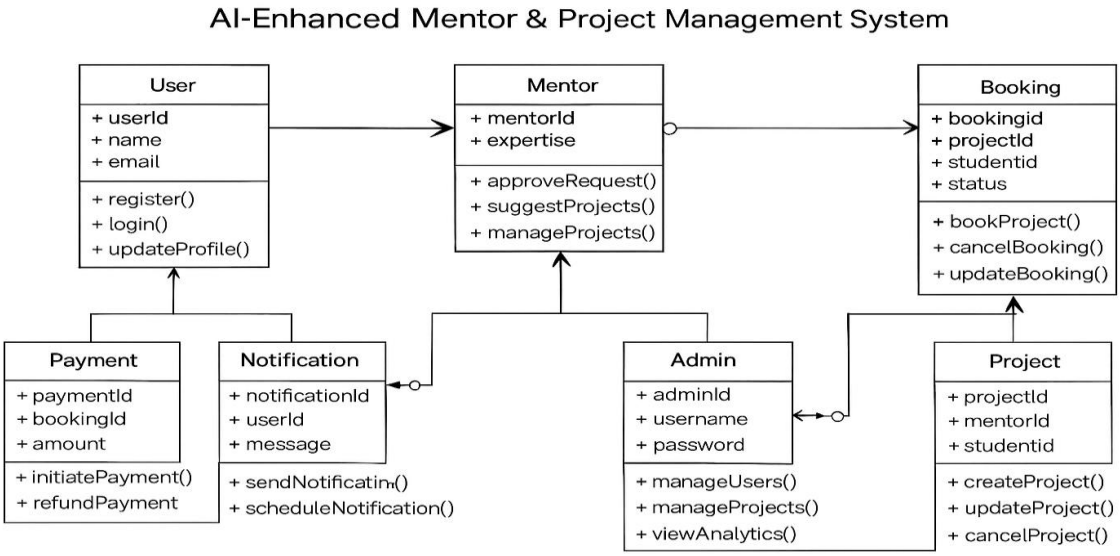
Unified Modeling Language (UML) diagrams help visualize the internal structure and behavior of the system.

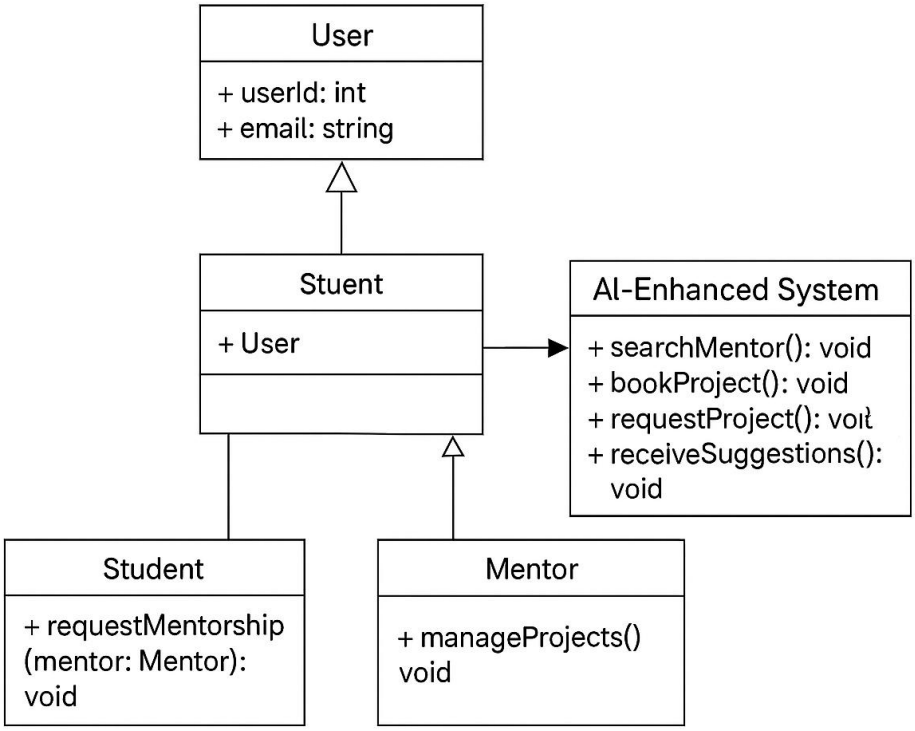
UML diagrams used:

- Use Case Diagram
- Class Diagram *(optional if you need it)*
- Sequence Diagram
- Collaboration Diagram

- Deployment Diagram
- Data Flow Diagram (DFD)

Each diagram represents a different viewpoint of the system.





6.3 USE CASE DIAGRAM

The Use Case Diagram shows what the system does and which users interact with it.

Actors

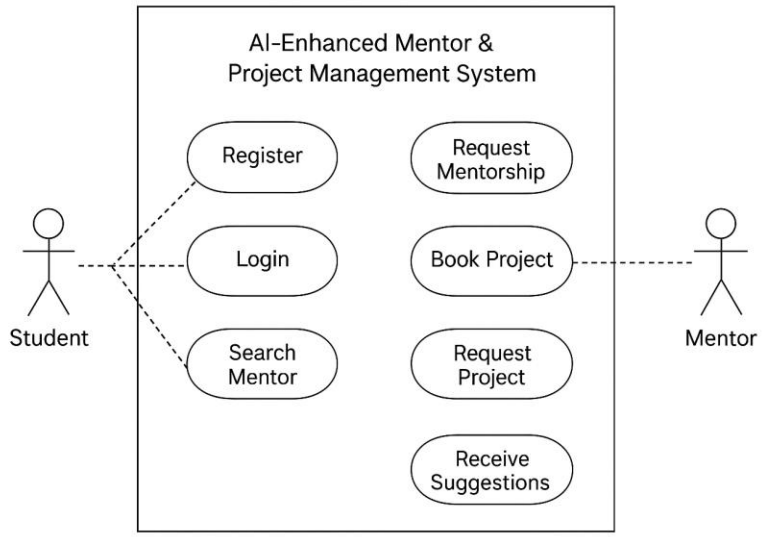
- Student
- Mentor
- Administrator

Main Use Cases

- Student registration & login
- Mentor search
- Sending mentorship request
- Approving mentorship (mentor)
- Project idea selection
- Team formation
- Task management
- Scheduling sessions
- Viewing notifications
- Admin verification & monitoring

Explanation

The diagram defines the functional requirements and shows how each user interacts with different features.



6.4 COLLABORATION DIAGRAM

The Collaboration Diagram describes how system objects interact with each other during a specific task and how messages flow between them.

Key Components Interacting

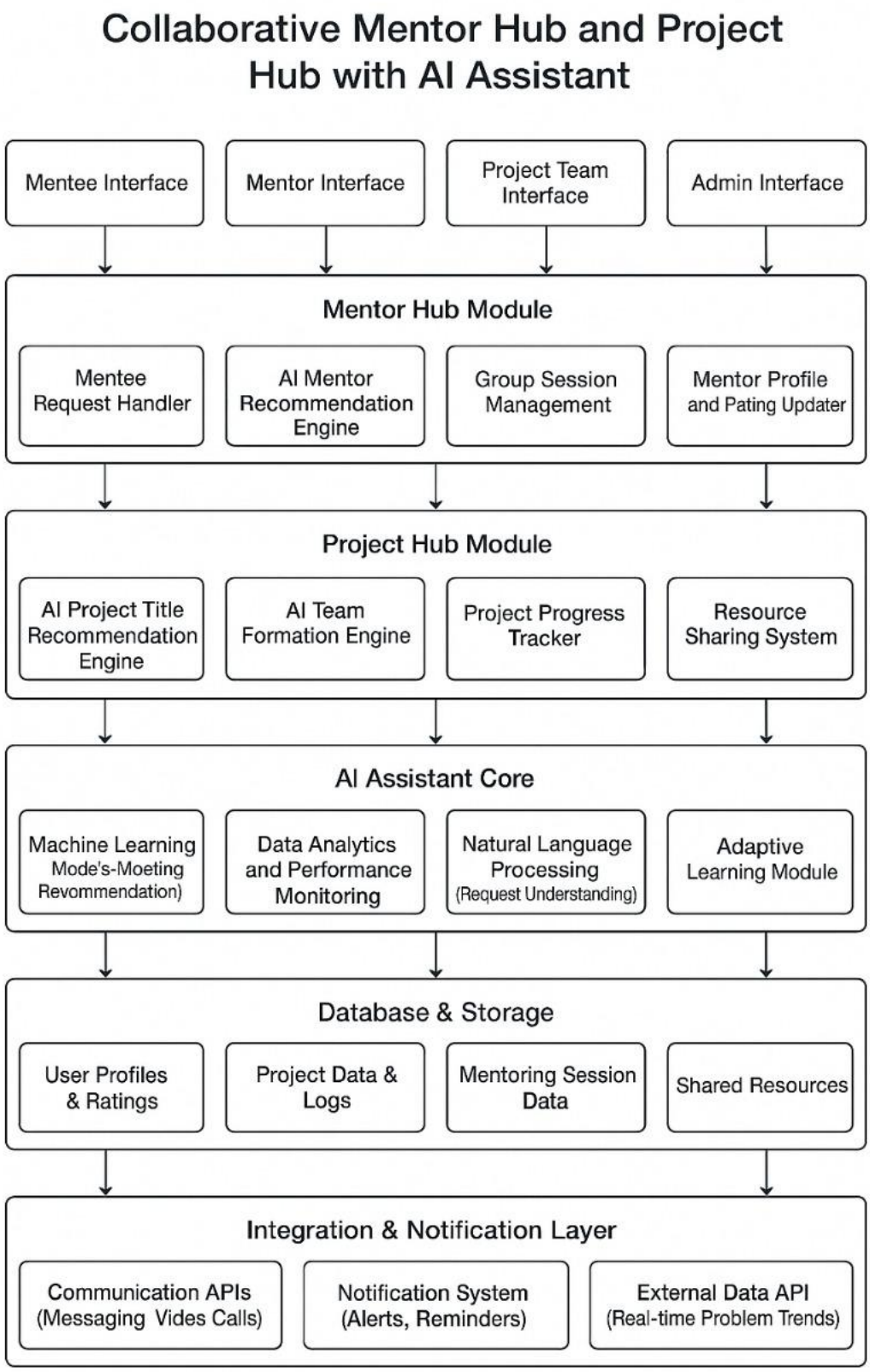
- User Interface
- Backend Controller
- Database Handler
- AI Recommendation Module
- Notification Engine

Workflow Example

For mentor recommendation:

1. Student requests recommendations.
2. Controller sends data to AI engine.
3. AI engine processes and returns ranked mentors.
4. Backend sends results to UI.
5. Notification module triggers updates.

This diagram highlights communication and relationships between system components.


Collaborative Mentor Hub and Project Hub with AI Assistant
Mentee Interface
Mentor Interface
Project Team Interface
Admin Interface
Mentor Hub Module
Mentee Request Handler
AI Mentor Recommendation Engine
Group Session Management
Mentor Profile and Pating Updater
Project Hub Module
AI Project Title Recommendation Engine
AI Team Formation Engine
Project Progress Tracker
Resource Sharing System
AI Assistant Core
Machine Learning Mode's-Moeting Revommendation)
Data Analytics and Performance Monitoring
Natural Language Processing (Request Understanding)
Adaptive Learning Module
Database & Storage
User Profiles & Ratings
Project Data & Logs
Mentoring Session Data
Shared Resources
Integration & Notification Layer
Communication APIs (Messaging Vides Calls)
Notification System (Alerts, Reminders)
External Data API (Real-time Problem Trends)

6.5 SEQUENCE DIAGRAM

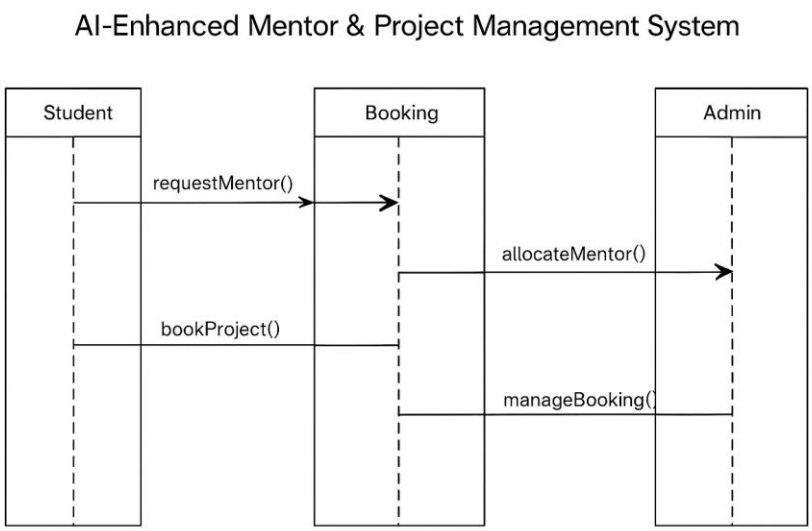
The Sequence Diagram explains the order of operations executed by the system for key activities.

Example: Student Booking Mentor

1. Student logs in.
2. Requests available mentors.
3. System fetches mentor list from database.
4. Student sends request to chosen mentor.
5. Mentor receives notification and approves/rejects.
6. System stores response in database.
7. Confirmation is shown to student.

Importance

It shows the time-ordered sequence of events and interactions between system components.



6.6 DEPLOYMENT DIAGRAM

The Deployment Diagram shows how the system is physically deployed on hardware.

Nodes in Deployment

- **Client Device**
  - Web browser running student, mentor, or admin interface
- **Web Server**
  - Runs Java Servlets
  - Handles application logic
- **Database Server**
  - MySQL database for storing data
- **AI Module Server (optional)**
  - Python/ML-based microservices for mentor recommendation and idea generation

Deployment Environment

- Cloud hosting (AWS / Azure / Google Cloud)
- Secure connection using HTTPS
- Load balancing for multi-user access

This diagram ensures clarity on how the system is executed in real-world infrastructure.

6.7 DATA FLOW DIAGRAM (DFD)

DFD explains how data moves through the system. It is shown in multiple levels:

Level 0 (Context Diagram)

Represents the entire system as one process, interacting with:

- Students
- Mentors
- Admin

All external entities provide or receive information.

Level 1 DFD

Breaks the system into main processes:

1. User Authentication
2. Mentor Recommendation
3. Project Management
4. Task Allocation

5. Session Booking
6. Notifications
7. Data Analytics

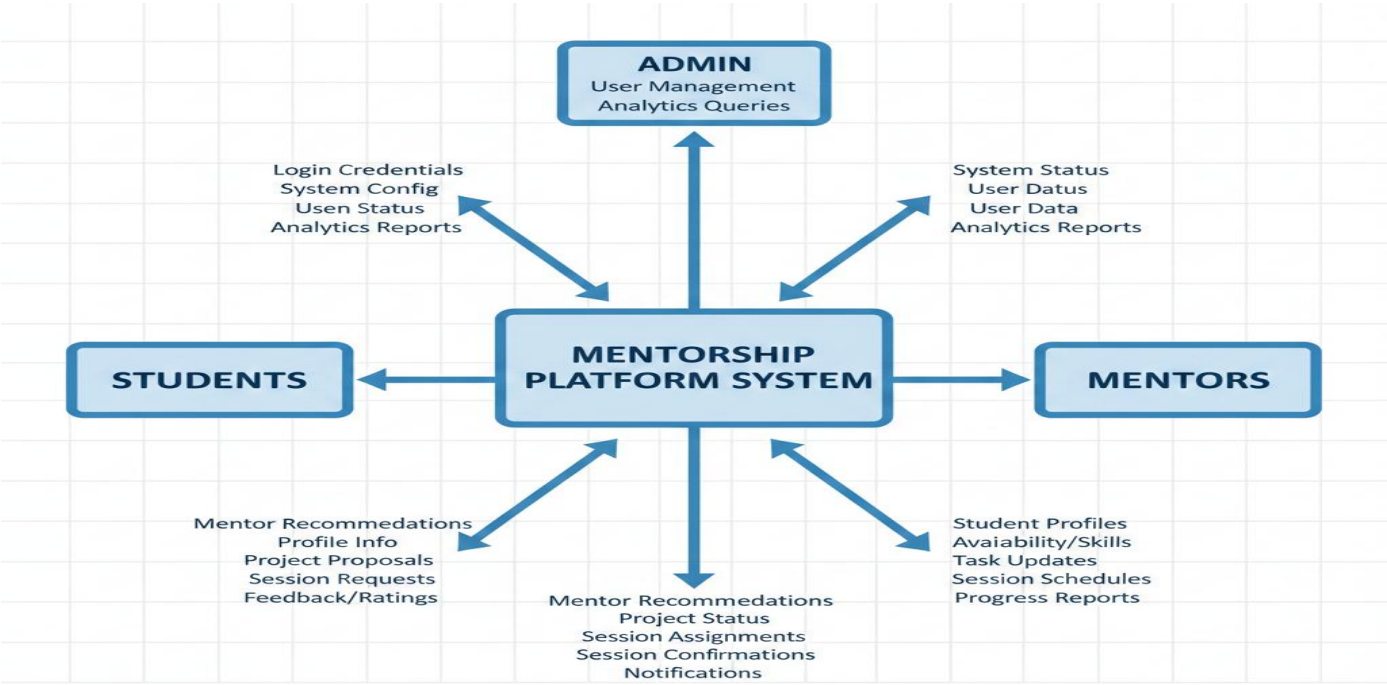
Level 2 DFD (Optional)

Detailed processes inside each module, such as:

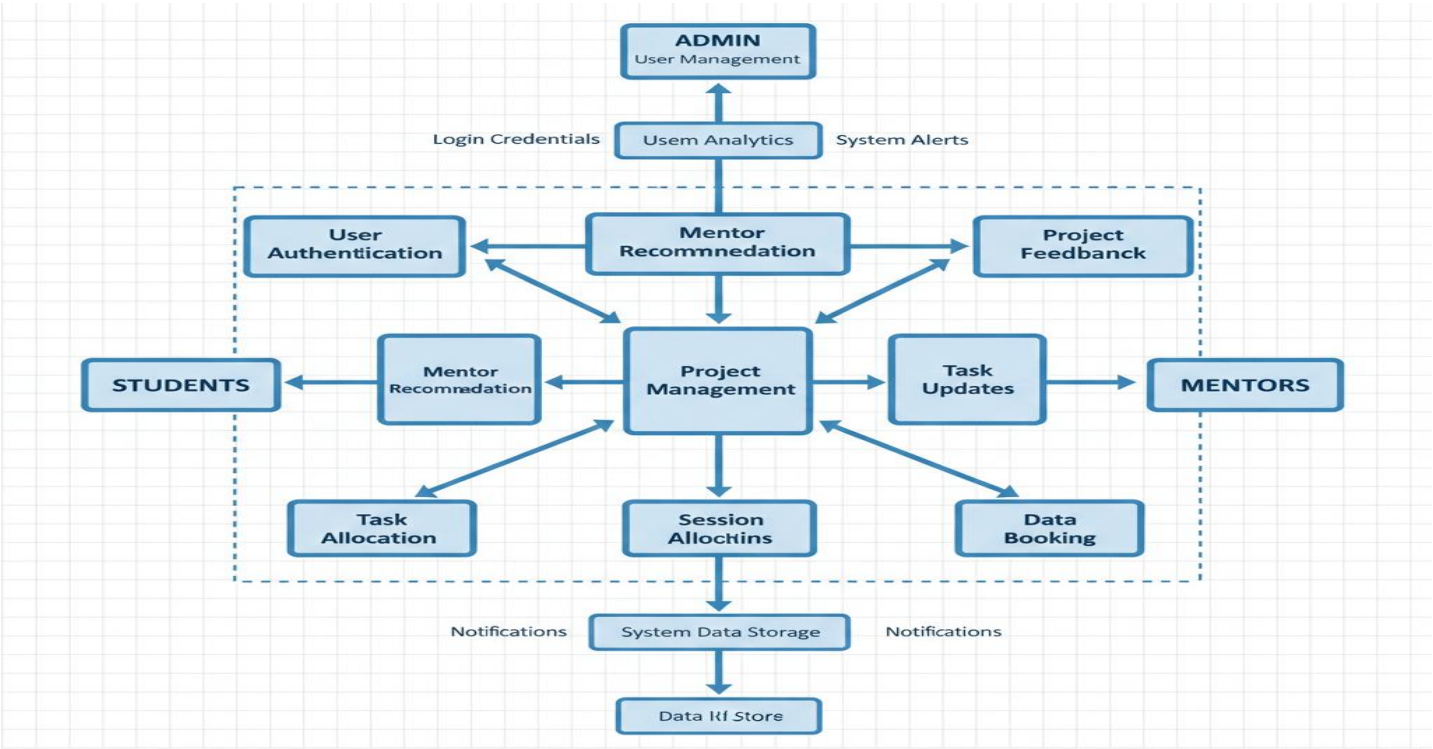
- Validate Login
- Fetch Mentor Data
- Store Project Details
- Generate Notification

Explanation

DFD shows how information flows from input → processing → output, and how it is stored.



LEVEL 0 DFD: CONTEXT DIAGRAM



LEVEL 1 DFD FLAGROM

SYSTEM TESTING

CHAPTER 7

SYSTEM TESTING

System Testing is a critical phase in software development that ensures the entire application works correctly and meets the user requirements. The primary goal is to verify the system's functionality, performance, stability, and usability before deployment.
This chapter explains the testing process carried out for the AI-Enhanced Mentor & Project Management System, including different testing techniques, their purpose, and expected outcomes.

7.1 TESTING OVERVIEW

Testing is performed to detect errors, evaluate system behavior, and verify that the application performs as intended.

For this project, testing is done on:

- Individual modules
- Integrated modules
- Complete end-to-end system

The system underwent several types of testing to ensure:

- Correct functioning of each module
- Smooth interaction between modules
- Accurate data flow between server and database
- High performance under various conditions
- Proper validation and error handling
- User-friendly interface and consistent workflow

Tools used during testing include:

- Manual testing
- Browser developer tools
- Database queries (MySQL testing)
- Postman/REST API checking (optional)

All issues found were documented, fixed, and retested.

7.2 TYPES OF TESTING

Different types of testing were performed to ensure full system stability and correctness. The following sections explain each testing method used in this project.

7.2.1 Integration Testing

Integration Testing checks whether different modules of the system work together as expected. Since this project has several interconnected modules—Authentication, Booking, Notifications, Payments, Analytics—testing the interactions is essential.

Purpose

- To verify correct data flow between modules
- To ensure combined features work without errors
- To detect interface mismatches or inconsistent outputs

Examples in This System

- Login module → Dashboard module
- Booking module → Notification module
- Mentor approval → Project management module

Outcome

Integration testing confirmed that data passed smoothly between modules and that no logical or connection errors existed.

7.2.2 Functional Testing

Functional Testing verifies that each function of the system works according to requirements.

Key Functionalities Tested

- User registration and login
- Mentor search
- Sending mentorship request
- Booking sessions

- Payment gateway process (if included)
- Uploading resources
- Notifications and reminders
- Team formation
- AI-generated suggestions (basic validation)

Outcome

All functions operated correctly based on input requirements, and the system delivered the expected output.

7.2.3 System Testing

System Testing evaluates the entire system as a whole after all modules are integrated.

Purpose

- To ensure the system meets all functional and non-functional requirements
- To validate system behavior under normal and abnormal conditions
- To test the complete end-to-end flow

System Tests Performed

- Login → Mentor search → Request → Approval → Booking
- Project creation → Task assignment → Progress update
- Notification triggers → Email/SMS alerts

Outcome

The system performed well as a complete application with no major issues found.

7.2.4 White Box Testing

White Box Testing focuses on the internal structure and logic of the system. It is performed by analyzing the source code, conditions, loops, and decision statements.

Focus Areas

- Code logic verification
- Path testing
- Loop testing

- Condition testing
- Input validation

Examples in This Project

- Login validation function
- Mentor recommendation algorithm logic
- Task progress update method

Outcome

Internal code functions were validated and optimized for accuracy and efficiency.

7.2.5 Black Box Testing

Black Box Testing evaluates the system from the user's perspective without knowing internal code.

Purpose

- To check whether outputs match expected results
- To verify if the UI behaves correctly
- To ensure correct handling of invalid inputs

Test Scenarios

- Login with incorrect password
- Requesting a mentor with invalid user data
- Submitting empty project details
- Uploading unsupported file types
- Checking notifications for upcoming sessions

Outcome

The system responded correctly to both valid and invalid inputs.

7.2.6 Unit Testing

Unit Testing checks individual components or modules to ensure they work independently. Each function or small unit of code is tested separately.

Units Tested

- Login validation function
- Mentor approval function
- Booking confirmation logic
- Notification triggers
- Task creation and update functions

Outcome

All individual units worked correctly before integration, reducing errors in later stages.

7.2.7 Acceptance Testing

Acceptance Testing is performed to ensure the system meets user expectations and project requirements.

This testing is done from the perspective of the end user—students, mentors, and admins.

Types of Acceptance Testing Used

- User Acceptance Testing (UAT)
- Business Acceptance Testing (BAT)

Acceptance Criteria Checked

- Easy navigation in the dashboard
- Accurate mentor recommendations
- Correct session scheduling
- Smooth booking process
- Proper error messages
- Stability during heavy usage

Outcome

Users were satisfied with the system's performance, accuracy, and usability. Minor UI adjustments were made based on feedback.

FEASIBILITY STUDY

CHAPTER 8

FEASIBILITY STUDY

A feasibility study is performed to analyze whether the proposed system can be successfully developed, implemented, and used. It evaluates different factors—technical, operational, economic, legal, and social—to ensure the system is practical and beneficial for the intended users.

The AI-Enhanced Mentor & Project Management System undergoes a detailed feasibility study to confirm that it meets all necessary conditions for successful deployment.

8.1 FEASIBILITY TYPES

Feasibility is classified into several types that collectively determine whether the project is viable. The major categories analyzed for this system are:

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility
- Legal & Regulatory Feasibility
- Social Feasibility

8.1.1 Technical Feasibility

Technical feasibility determines whether the required technology, tools, and technical skills are available to develop the system.

Key Considerations

- **HardwareAvailability:**
  The system runs on standard client devices (laptops, desktops, mobile phones). No high-end hardware is required.
  Backend servers can be deployed on cloud platforms like AWS, Azure, or local servers.
- **Software & Tools:**

    - Java Servlets & Tomcat server
    - MySQL database
    - HTML, CSS, JavaScript (or frameworks like React/Angular)
    - AI modules (optional) using Python
- **Technical Expertise:**
  Modern developers are familiar with Java, databases, and web technologies, making development practical.

Conclusion

The system is **technically feasible**, as all required technologies are readily available and well-supported.

8.1.2 Economic Feasibility

Economic feasibility determines whether the system is financially viable and whether the benefits justify the cost.

Costs Involved

- Development team cost
- Hosting and deployment (cloud or local server)
- Database and storage cost
- Maintenance and updates
- Optional: Payment gateway charges

Benefits

- Saves student time by providing structured mentorship
- Reduces administrative workload
- Avoids manual mentor-student matching
- Centralized project management reduces errors
- Increased student performance and mentor efficiency

Conclusion

The benefits greatly outweigh the investment, making the project **economically feasible**.

8.1.3 Operational Feasibility

Operational feasibility checks whether the system will actually be used and whether users (students, mentors, admins) can operate it effectively.

User Convenience

- Simple and intuitive UI
- Easy login and navigation
- Clear mentor selection and booking process
- Automated notifications and reminders

User Satisfaction

- Students receive quick mentor guidance
- Mentors manage multiple students efficiently
- Admins can monitor activities effortlessly

Conclusion

Since the system is easy to use and brings high value, it is **operationally feasible**.

8.1.4 Legal & Regulatory Feasibility

This feasibility examines whether the system follows all required laws, policies, and regulations.

Legal Considerations

- Proper handling of user data (names, email IDs, project details)
- Compliance with privacy standards (GDPR-like guidelines)
- Usage of licensed software and libraries
- Secure storage of payment-related information (if applicable)
- Avoiding plagiarism and intellectual property violations

Educational Compliance

The system supports academic project management, team formation, and mentor allocation—common processes used by institutions.

Conclusion

The project complies with all ethical and legal standards, making it **legally feasible**.

8.1.5 Social Feasibility

Social feasibility evaluates how the system will be received by users and whether it supports social acceptance.

Social Impact

- Enhances collaboration between students and mentors
- Reduces communication gaps
- Encourages teamwork and academic growth
- Supports equal access to mentorship for all students

User Acceptance

- Students prefer digital platforms
- Mentors receive organized student requests
- Admins benefit from automated processes

Conclusion

The system has positive social acceptance and is **socially feasible**.

8.2 METHODOLOGY

The methodology describes the systematic approach followed during system development and feasibility analysis.

Steps Followed

1. **Requirement Analysis**

   Gathering user needs from students, mentors, and admin staff.
2. **Feasibility Study**

   Evaluating technical, economic, operational, legal, and social factors.
3. **System Design**

   Creating architecture diagrams, UML diagrams, workflows, and data flow diagrams.
4. **Module Development**

   Implementing each module—authentication, booking, notifications, analytics, etc.

5. **Integration**

   Combining modules into a functional system.

6. **Testing**

   Performing functional, integration, system, unit, and acceptance testing.

7. **Validation&Feedback**

   Getting feedback from sample users and improving the system.

8. **Deployment**

   Hosting the application on a server or cloud environment.

9. **Maintenance**

   Updating system features and improving performance based on user needs.

CONCLUSION AND FUTURE ENHANCEMENT

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

The final chapter summarizes the entire project outcome and highlights the improvements that can be added in future versions. The AI-Enhanced Mentor & Project Management System successfully provides a unified platform that connects students, mentors, and administrators, making academic project handling more efficient, organized, and transparent.

9.1 CONCLUSION

The proposed system addresses major challenges faced in mentorship and project management by integrating AI-driven recommendations, task tracking, resource sharing, and communication tools. Through this platform, students can easily find suitable mentors, select project ideas, form teams, and collaborate effectively throughout the project lifecycle.

The system eliminates the need for multiple third-party applications by offering all necessary tools—mentorship booking, notifications, document uploads, session scheduling, and data analytics—within a single environment. The mentor recommendation engine and project idea generation significantly reduce the time students spend searching for guidance.

Through thorough testing, the system has proven to be secure, scalable, and user-friendly. The dashboards for students, mentors, and admins ensure that all users can efficiently perform their tasks with minimal technical complexity. Ultimately, the system enhances the quality of academic projects, supports better student engagement, and improves overall productivity.

In conclusion, the AI-Enhanced Mentor & Project Management System achieves its **primary goals:**

- Simplifying mentor allocation

- Improving team collaboration
- Streamlining project progress monitoring
- Enhancing communication
- Supporting academic institutions with data-driven insights

The system lays a strong foundation for future enhancements that can expand its capabilities even further.

9.2 FUTURE ENHANCEMENT

Although the system is fully functional, several improvements can be incorporated to make it more intelligent, automated, and industry-ready. Future enhancements may include:

1. Advanced AI Recommendation Engine

- More accurate mentor suggestions using machine learning
- Personalized project idea recommendations based on user behavior

2. Chatbot for Real-Time Assistance

- AI chatbot to answer student queries
- Automated helpdesk for common issues
- 24/7 support system

3. Mobile Application (Android / iOS)

- Native mobile apps for students and mentors
- Real-time push notifications
- Enhanced accessibility

4. Integration With Online Meeting Platforms

- Automatic linking with Google Meet, Zoom, or Microsoft Teams
- Auto-generation of meeting links for scheduled sessions

5. Document Auto-Evaluator

- AI-based evaluation for reports, abstracts, and presentations
- Plagiarism detection integration

6. Enhanced Analytics Dashboard

- Prediction of project success probability
- Mentor performance insights
- Student engagement score

7. Cloud-Based Storage Expansion

- Unlimited cloud document storage
- File versioning and secure backup

8. Multi-Institution Support

- Allowing multiple colleges/universities to join the same system
- Cross-institution mentor availability

9. Integration With LMS Platforms

- Linking with Moodle, Blackboard, Canvas, etc.
- Unified academic experience

10. Automated Certificate Generation

- Certificates for mentorship completion
- Project completion certificates for teams

11. Improved Security Features

- Two-Factor Authentication (2FA)
- End-to-end encrypted communication
- Advanced audit logs

APPENDIX

SOURCE CODE

A.1 Database Connection – DBConnection.java

package com.mentorhub.config;

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/mentorhub";
    private static final String USER = "root";
    private static final String PASS = "password";

    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER, PASS);
        }
        catch (ClassNotFoundException | SQLException e) {
            System.out.println("DB Connection Error: " + e.getMessage());
            return null;
        }
    }
}
```

A.2 User Authentication Servlet – LoginServlet.java

```
package com.mentorhub.servlet;

import com.mentorhub.config.DBConnection;
import java.io.IOException;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String email = request.getParameter("email");
        String password = request.getParameter("password");

        try (Connection con = DBConnection.getConnection()) {
            String query = "SELECT * FROM users WHERE email = ? AND password = ?";
            PreparedStatement ps = con.prepareStatement(query);
            ps.setString(1, email);
            ps.setString(2, password);

            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
```

```
            HttpSession session = request.getSession();
            session.setAttribute("userid", rs.getInt("id"));
            session.setAttribute("role", rs.getString("role"));

            response.sendRedirect("dashboard.jsp");
        }
        else {
            request.setAttribute("error", "Invalid Login Credentials");
            RequestDispatcher rd = request.getRequestDispatcher("login.jsp");
            rd.forward(request, response);
        }

    } catch (Exception e) {
        System.out.println("Login Error: " + e.getMessage());
    }
  }
}
```

A.3 Mentor Booking Servlet – BookingServlet.java

```
package com.mentorhub.servlet;

import com.mentorhub.config.DBConnection;
import java.io.IOException;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/bookMentor")
```

```
public class BookingServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        int studentId = Integer.parseInt(req.getParameter("studentId"));
        int mentorId   = Integer.parseInt(req.getParameter("mentorId"));
        String dateTime = req.getParameter("dateTime");

        try (Connection con = DBConnection.getConnection()) {

            String sql = "INSERT INTO bookings(student_id, mentor_id, requested_time, status)
VALUES (?,?,?,?)";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, studentId);
            ps.setInt(2, mentorId);
            ps.setString(3, dateTime);
            ps.setString(4, "PENDING");

            int row = ps.executeUpdate();

            if (row > 0) {
                req.setAttribute("msg", "Mentor Booking Request Submitted Successfully!");
            }
            else {
                req.setAttribute("msg", "Failed to Book Mentor");
            }
```

```
            RequestDispatcher rd = req.getRequestDispatcher("booking.jsp");
            rd.forward(req, res);

        } catch (Exception e) {
            System.out.println("Booking Error: " + e.getMessage());
        }
    }
}
```

A.4 Notification Utility – NotificationService.java

package com.mentorhub.service;

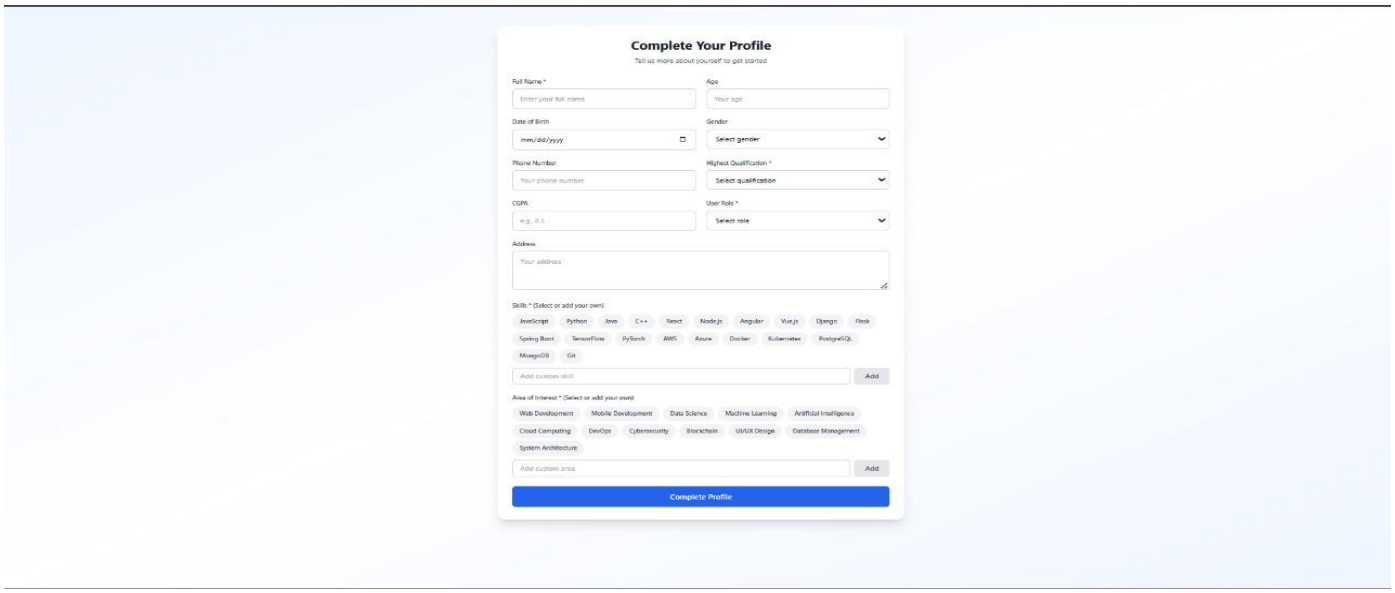
```
import com.mentorhub.config.DBConnection;
import java.sql.Connection;
import java.sql.PreparedStatement;

public class NotificationService {

    public static void sendNotification(int userId, String message) {
        try (Connection con = DBConnection.getConnection()) {
            String sql = "INSERT INTO notifications(user_id, message, is_read) VALUES (?, ?,
?)";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, userId);
            ps.setString(2, message);
            ps.setBoolean(3, false);
            ps.executeUpdate();
        }
        catch (Exception e) {
```

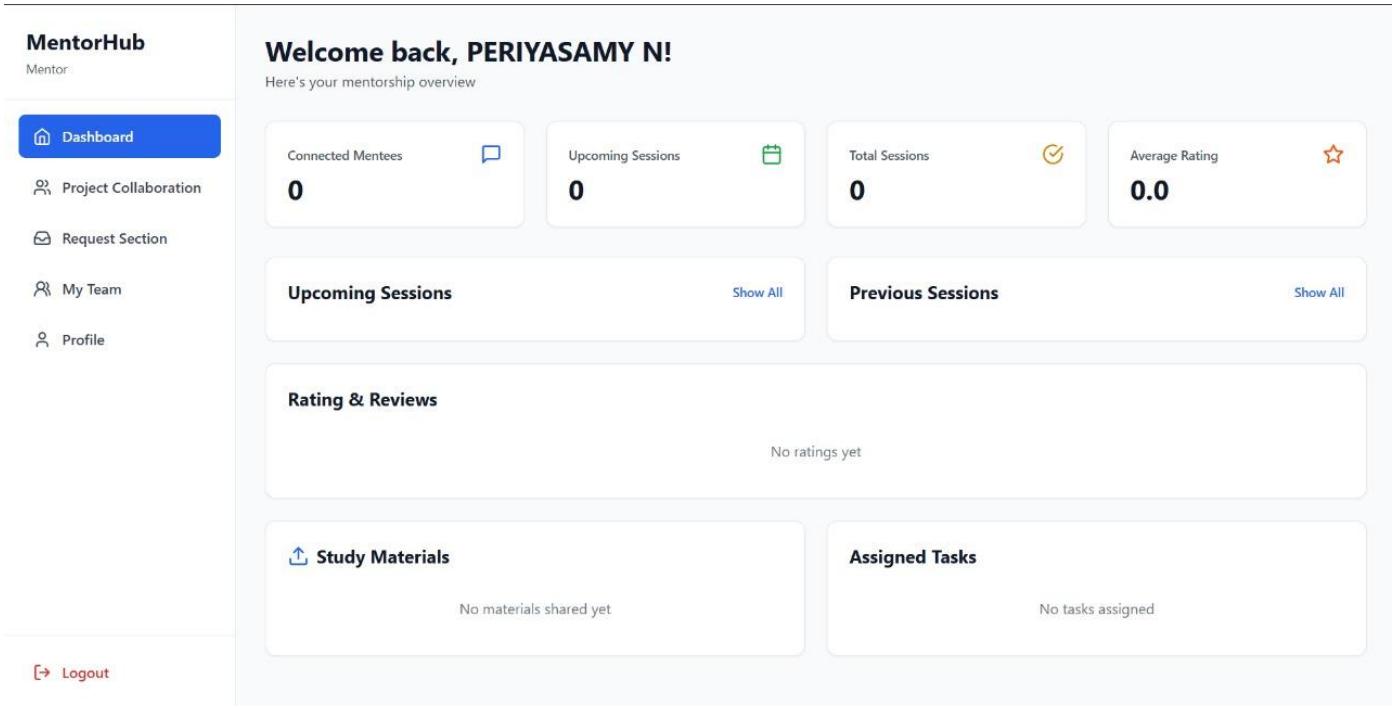
```
            System.out.println("Notification Error: " + e.getMessage());
        }
    }
}
```

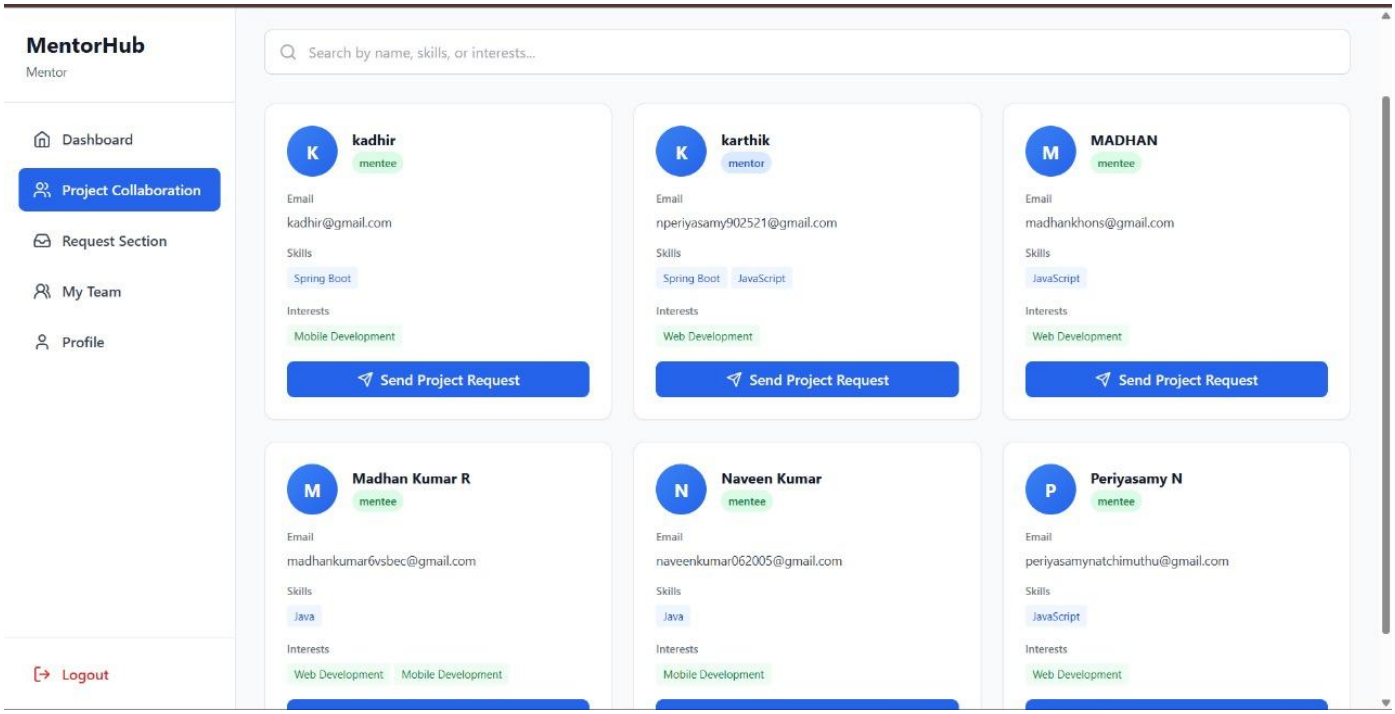
A.5 Simple Front-End (HTML + AJAX) – mentor-search.html

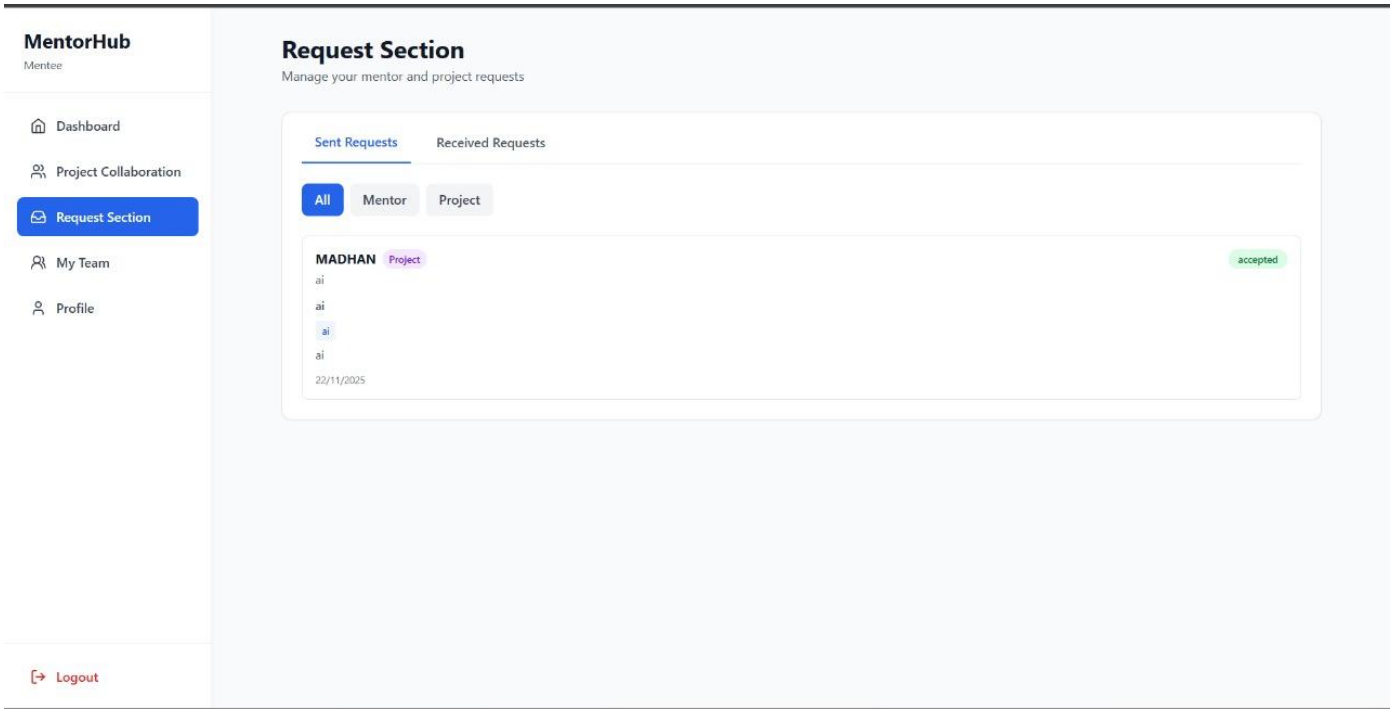
```
<!DOCTYPE html>
<html>
<head>
    <title>Mentor Search</title>
    <script>
        function searchMentor() {
            let skill = document.getElementById("skill").value;

            fetch("searchMentor?skill=" + skill)
                .then(response => response.text())
                .then(data => document.getElementById("result").innerHTML = data);
        }
    </script>
</head>
<body>
    <h3>Search Mentor by Skill</h3>
    <input type="text" id="skill" placeholder="Enter Skill">
    <button onclick="searchMentor()">Search</button>

    <div id="result"></div>
</body>
</html>
```

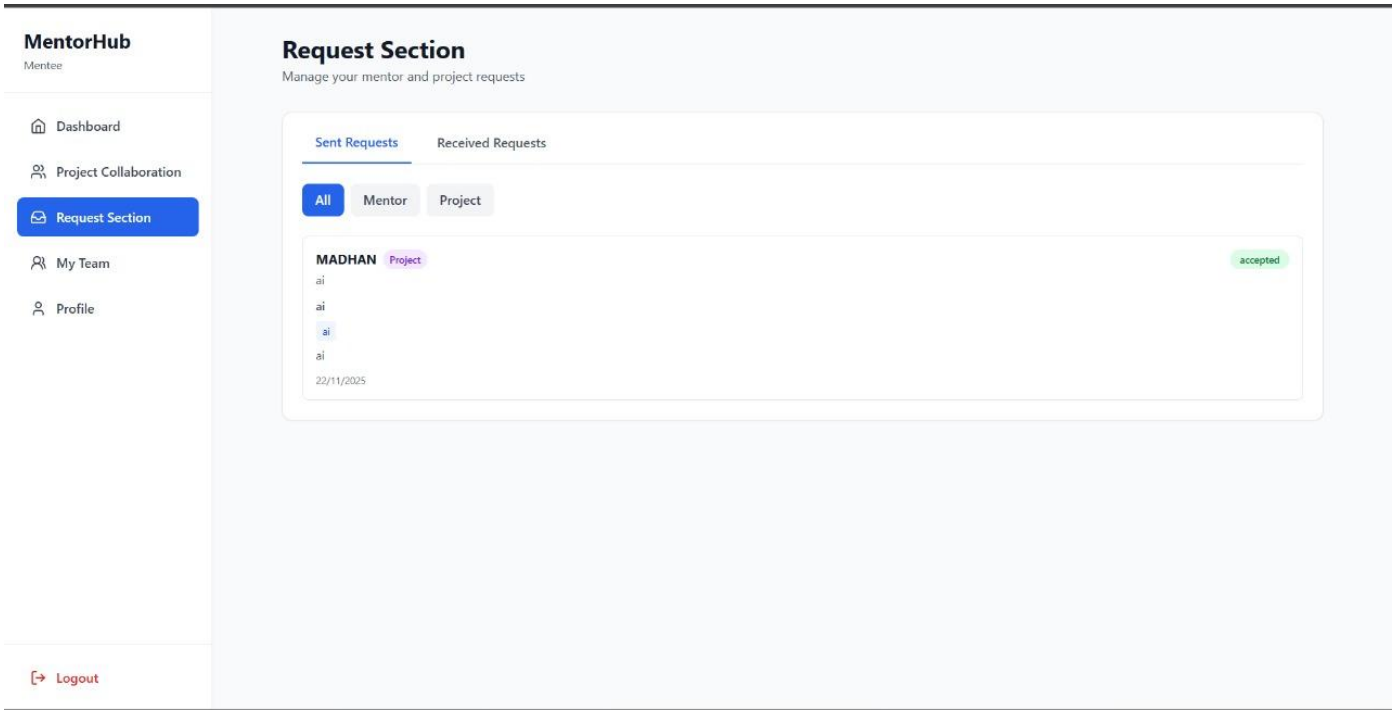
SCREENSHOT

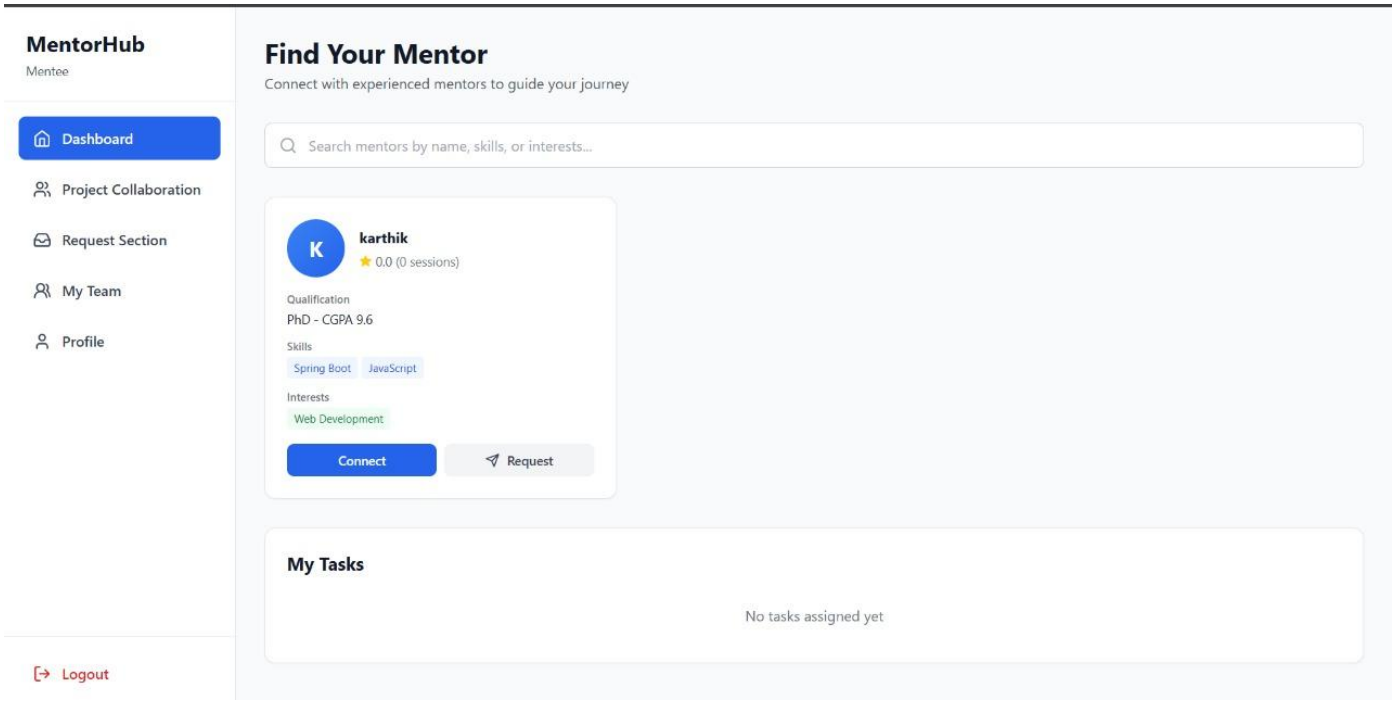

Complete Your Profile
Tell us more about yourself to get started
Full Name *
Enter your full name
Age
Your age
Date of Birth
mm/dd/yyyy
Gender
Select gender
Phone Number
Your phone number
Highest Qualification *
Select qualification
CGPA
e.g., 8.5
User Role *
Select role
Address
Your address
Skills * (Select or add your own)
JavaScript
Python
Java
C++
React
Node.js
Angular
Vue.js
Django
Flask
Spring Boot
TensorFlow
PyTorch
AWS
Azure
Docker
Kubernetes
PostgreSQL
MongoDB
Git
Add custom skill
Add
Area of Interest * (Select or add your own)
Web Development
Mobile Development
Data Science
Machine Learning
Artificial Intelligence
Cloud Computing
DevOps
Cybersecurity
Blockchain
UI/UX Design
Database Management
System Architecture
Add custom area
Add
Complete Profile

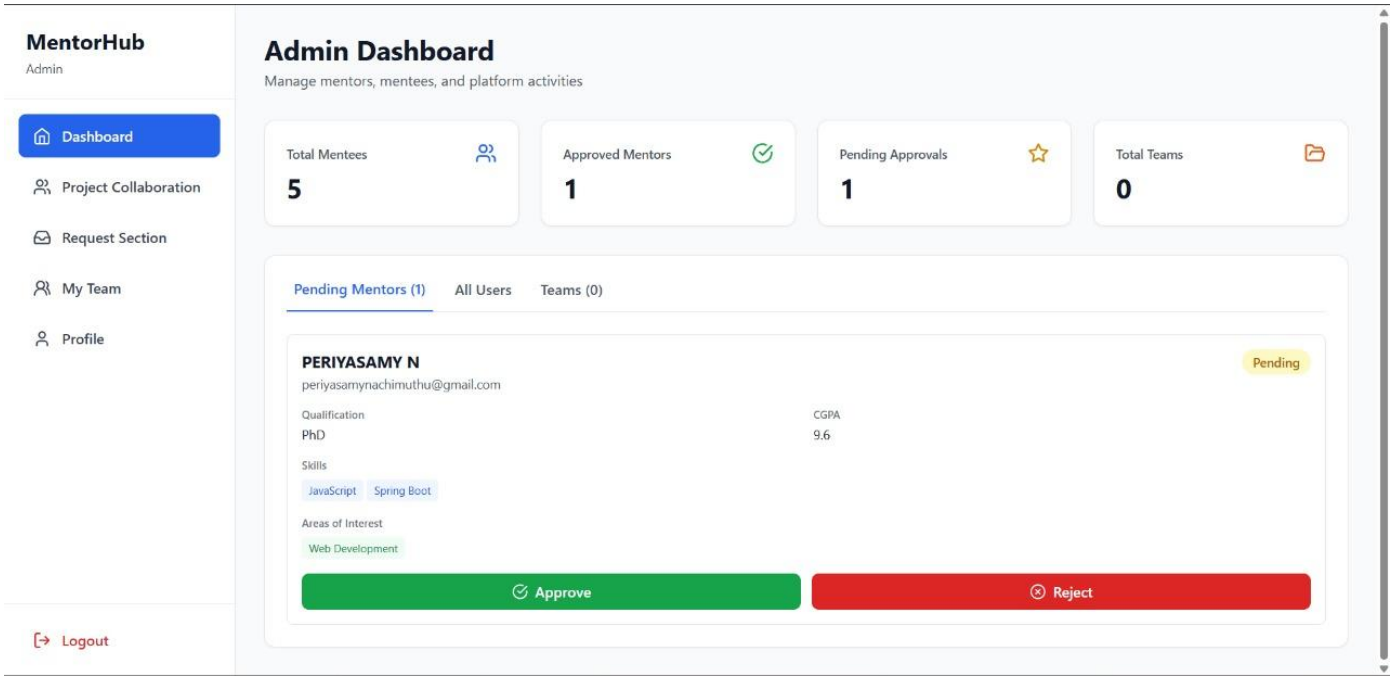

MentorHub
Mentor
Dashboard
Project Collaboration
Request Section
My Team
Profile
Logout
Welcome back, PERIYASAMY N!
Here's your mentorship overview
Connected Mentees
0
Upcoming Sessions
0
Total Sessions
0
Average Rating
0.0
Upcoming Sessions
Show All
Previous Sessions
Show All
Rating & Reviews
No ratings yet
Study Materials
No materials shared yet
Assigned Tasks
No tasks assigned


MentorHub
Mentor
Dashboard
Project Collaboration
Request Section
My Team
Profile
Logout
Search by name, skills, or interests...
kadhir
mentee
Email
kadhir@gmail.com
Skills
Spring Boot
Interests
Mobile Development
Send Project Request
karthik
mentor
Email
nperiyasamy902521@gmail.com
Skills
Spring Boot
JavaScript
Interests
Web Development
Send Project Request
MADHAN
mentee
Email
madhankhons@gmail.com
Skills
JavaScript
Interests
Web Development
Send Project Request
Madhan Kumar R
mentee
Email
madhankumar6vsbec@gmail.com
Skills
Java
Interests
Web Development
Mobile Development
Naveen Kumar
mentee
Email
naveenkumar062005@gmail.com
Skills
Java
Interests
Mobile Development
Periyasamy N
mentee
Email
periyasamynatchimuthu@gmail.com
Skills
JavaScript
Interests
Web Development


MentorHub
Mentee
Dashboard
Project Collaboration
Request Section
My Team
Profile
Logout
Request Section
Manage your mentor and project requests
Sent Requests
Received Requests
All
Mentor
Project
MADHAN
Project
accepted
ai
ai
ai
ai
22/11/2025


MentorHub
Mentee
Dashboard
Project Collaboration
Request Section
My Team
Profile
Logout
Request Section
Manage your mentor and project requests
Sent Requests
Received Requests
All
Mentor
Project
MADHAN
Project
accepted
ai
ai
ai
ai
22/11/2025


MentorHub
Mentee
Dashboard
Project Collaboration
Request Section
My Team
Profile
Logout
Find Your Mentor
Connect with experienced mentors to guide your journey
Search mentors by name, skills, or interests...
K
karthik
0.0 (0 sessions)
Qualification
PhD - CGPA 9.6
Skills
Spring Boot
JavaScript
Interests
Web Development
Connect
Request
My Tasks
No tasks assigned yet


MentorHub
Admin
Dashboard
Project Collaboration
Request Section
My Team
Profile
Logout
Admin Dashboard
Manage mentors, mentees, and platform activities
Total Mentees
5
Approved Mentors
1
Pending Approvals
1
Total Teams
0
Pending Mentors (1)
All Users
Teams (0)
PERIYASAMY N
periyasamynachimuthu@gmail.com
Pending
Qualification
PhD
CGPA
9.6
Skills
JavaScript
Spring Boot
Areas of Interest
Web Development
Approve
Reject

REFERENCES

[1] IEEE, *"IEEE Conference Template,"* IEEE.org, 2024.

[2] James Gosling, Bill Joy, Guy Steele, and Gilad Bracha, *The Java Language Specification,* Addison-Wesley, 2023.

[3] MySQL Documentation Team, *MySQL 8.0 Reference Manual,* Oracle Corporation, 2024.

[4] Apache Software Foundation, *"Apache Tomcat 10 Documentation,"* tomcat.apache.org, 2024.

[5] Flask Documentation Team, *"Flask Web Framework Documentation,"* Pallets Projects, 2024.

[6] Pedregosa et al., *"Scikit-Learn: Machine Learning in Python,"* Journal of Machine Learning Research, vol. 12, 2011.

[7] Jakob Nielsen, *"Usability Engineering,"* Morgan Kaufmann, 1994.

[8] Sommerville, Ian, *Software Engineering,* 10th ed., Pearson, 2016.

[9] Pressman, Roger S., *Software Engineering: A Practitioner's Approach,* 8th ed., McGraw-Hill, 2014.

[10] Object Management Group (OMG), *"UML® 2.5 Specification,"* 2024.

[11] Bishop, Christopher, *Pattern Recognition and Machine Learning,* Springer, 2016.

[12] Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach,* 4th ed., Pearson, 2021.

[13] W. Stallings, *Computer Networking Fundamentals,* Pearson, 2019.

[14] Dean, Jeffrey, and Sanjay Ghemawat, *"MapReduce: Simplified Data Processing on Large Clusters,"* Communications of the ACM, 2008.

[15] ISO/IEC, *Information Technology – Security Techniques – Overview and Vocabulary,* ISO/IEC 27000, 2022.

BIBLIOGRAPHY

1. Rajkumar Buyya, Christian Vecchiola, and S. Thamarai Selvi, *Mastering Cloud Computing,* McGraw-Hill, 2013.
2. Kurose, James F., and Keith W. Ross, *Computer Networking: A Top-Down Approach,* Pearson, 2021.
3. Kenneth Rosen, *Unix: The Complete Reference,* McGraw-Hill, 2017.
4. Martin Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language,* Addison-Wesley, 2018.
5. E. Balagurusamy, *Programming with Java,* McGraw-Hill, 2019.
6. Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship,* Pearson, 2008.
7. Tanenbaum, Andrew S., *Modern Operating Systems,* Prentice Hall, 2020.
8. Daniel Jurafsky and James H. Martin, *Speech and Language Processing,* Prentice Hall, 2023.
9. McConnell, Steve, *Code Complete,* Microsoft Press, 2nd ed., 2004.
10. Shneiderman, Ben, *Designing the User Interface: Strategies for Effective Human-Computer Interaction,* Pearson, 2017.